Lab_Assignment_11.1

Name : D.Rishwanth
Roll No : 2503A52395

Topic : DATA STRUCTURES WITH AI:
Implementing Fundamental Structures

Lab Objectives

1 .Use AI to assist in designing and implementing fundamental data structures in Python.

2 .Learn how to prompt AI for structure creation, optimization, and documentation.

3 .Improve understanding of Lists, Stacks, Queues, Linked Lists, Trees, Graphs, and Hash Tables.

4 .Enhance code quality with AI-generated comments and performance suggestions.

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

class Stack:pass

Expected Output:

 A functional stack implementation with all required methods and docstrings.

Input:

```
class Stack:
    """A simple stack implementation using a Python list."""

    def __init__(self):
        """Initialize an empty stack."""
        self.items = []
```

```
    def push(self, item):
        """Add an item to the top of the stack."""
        self.items.append(item)

    def pop(self):
        """Remove and return the top item from the stack."""
        if self.is_empty():
            raise IndexError("Cannot pop from an empty stack.")
        return self.items.pop()

    def peek(self):
        """Return the top item without removing it."""
        if self.is_empty():
            raise IndexError("Cannot peek at an empty stack.")
        return self.items[-1]

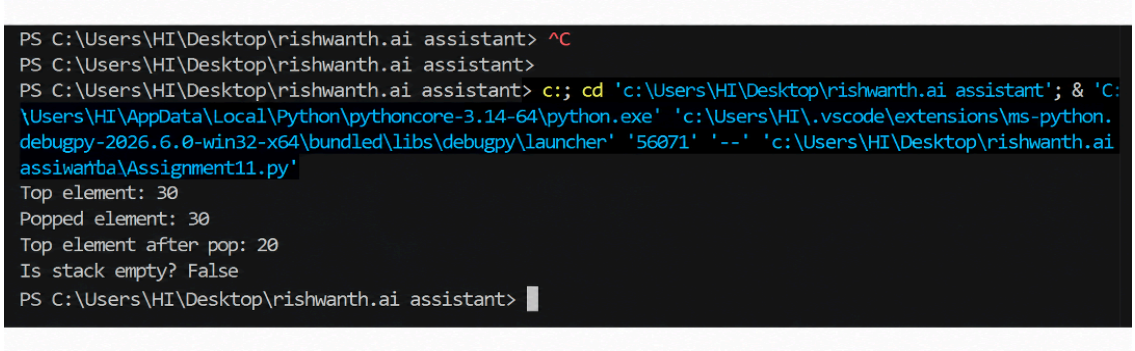
    def is_empty(self):
        """Return True if the stack is empty."""
        return len(self.items) == 0


# Example usage
stack = Stack()

stack.push(10)
stack.push(20)
stack.push(30)

print("Top element:", stack.peek())
print("Popped element:", stack.pop())
print("Top element after pop:", stack.peek())
print("Is stack empty?", stack.is_empty())
```

Output:



Explanation :-

A Stack is a data structure that follows the LIFO (Last In, First Out) principle. In this implementation, a Python list is used to store the elements. The push() method adds an element to the top of the stack, pop() removes the top element, peek() returns the top element without removing it, and is_empty() checks whether the stack is empty.

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

class Queue:

pass

Expected Output:

 FIFO-based queue class with enqueue, dequeue, peek, and size methods.

Input :

```
class Queue:
    """A simple queue implementation using a Python list."""

    def __init__(self):
        """Initialize an empty queue."""
        self.items = []

    def enqueue(self, item):
        """Add an item to the rear of the queue."""
        self.items.append(item)

    def dequeue(self):
        """Remove and return the item from the front of the queue."""
        if self.is_empty():
            raise IndexError("Cannot dequeue from an empty queue.")
        return self.items.pop(0)

    def peek(self):
        """Return the front item without removing it."""
        if self.is_empty():
            raise IndexError("Cannot peek at an empty queue.")
        return self.items[0]

    def size(self):
        """Return the number of items in the queue."""
```

```
        return len(self.items)

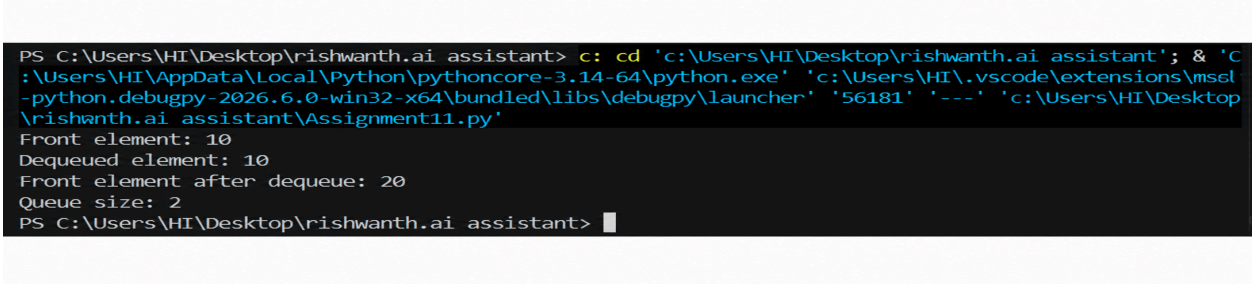
    def is_empty(self):
        """Return True if the queue is empty."""
        return len(self.items) == 0


# Example usage
queue = Queue()

queue.enqueue(10)
queue.enqueue(20)
queue.enqueue(30)

print("Front element:", queue.peek())
print("Dequeued element:", queue.dequeue())
print("Front element after dequeue:", queue.peek())
print("Queue size:", queue.size())
```

Output :



Explanation :-

A Queue is a data structure that follows the FIFO (First In, First Out) principle. In this implementation, a Python list is used to store the elements. The enqueue() method adds an element to the rear of the queue, dequeue() removes the element from the front, peek() returns the front element without removing it, and size() returns the number of elements in the queue.

Task Description #3 – Linked List
Task: Use AI to generate a Singly Linked List with insert and display
methods.

Sample Input Code:
class Node:
pass

Week6 -
Monday

class LinkedList:
pass
Expected Output:
✅ A working linked list implementation with clear method documentation

Input :-

```
class Node:
    """A node in a singly linked list."""

    def __init__(self, data):
        """Initialize the node with data."""
        self.data = data
        self.next = None


class LinkedList:
    """A simple singly linked list."""

    def __init__(self):
        """Initialize an empty linked list."""
        self.head = None

    def insert(self, data):
        """Insert a new node at the end of the linked list."""
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            return

        current = self.head

        while current.next is not None:
```

```
            current = current.next

        current.next = new_node

    def display(self):
        """Display all elements of the linked list."""
        current = self.head

        while current is not None:
            print(current.data, end=" -> ")
            current = current.next

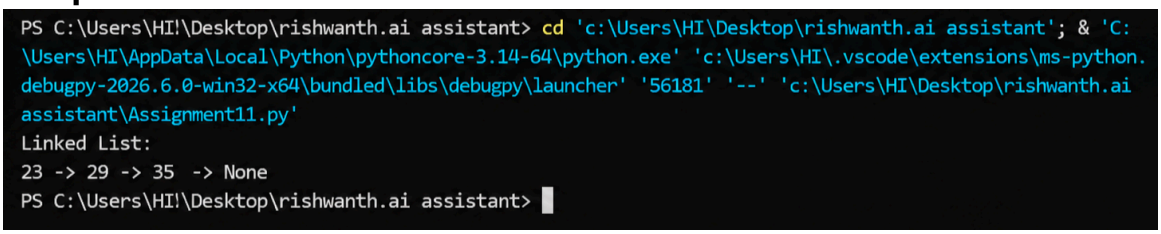
        print("None")


# Example usage
linked_list = LinkedList()

linked_list.insert(10)
linked_list.insert(20)
linked_list.insert(30)

print("Linked List:")
linked_list.display()
```

Output :-



Explanation :-

A Singly Linked List consists of a sequence of nodes, where each node contains data and a reference to the next node. The Node class is used to create individual nodes, while the LinkedList class manages the list. The insert() method adds a new node to the end of the list, and the display() method traverses the list and displays all the elements.

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

class BST:

pass

Expected Output:

✅ BST implementation with recursive insert and traversal methods.

Input :-

```
class Node:
    """A node in a Binary Search Tree."""

    def __init__(self, data):
        """Initialize the node with data."""
        self.data = data
        self.left = None
        self.right = None


class BST:
    """A Binary Search Tree implementation."""

    def __init__(self):
        """Initialize an empty Binary Search Tree."""
        self.root = None

    def insert_recursive(self, root, data):
        """Recursively insert data into the Binary Search Tree."""
        if root is None:
            return Node(data)

        if data < root.data:
            root.left = self.insert_recursive(root.left, data)
        elif data > root.data:
            root.right = self.insert_recursive(root.right, data)

        return root

    def insert(self, data):
```

```
        """Insert data into the Binary Search Tree."""
        self.root = self.insert_recursive(self.root, data)

    def inorder_recursive(self, root):
        """Recursively perform in-order traversal."""
        if root is not None:
            self.inorder_recursive(root.left)
            print(root.data, end=" ")
            self.inorder_recursive(root.right)

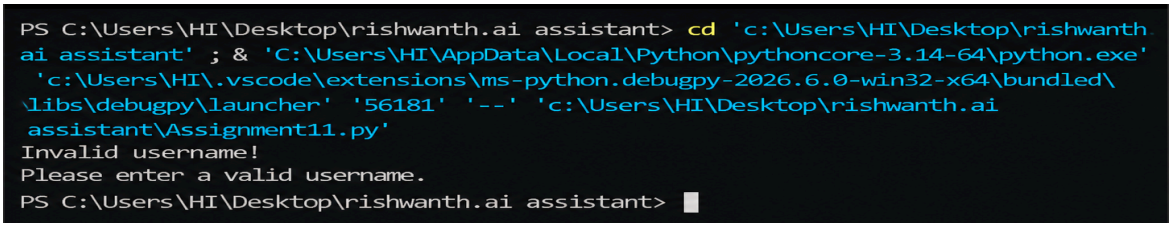
    def inorder(self):
        """Display the BST using in-order traversal."""
        self.inorder_recursive(self.root)


# Example usage
bst = BST()

bst.insert(50)
bst.insert(30)
bst.insert(70)
bst.insert(20)
bst.insert(40)
bst.insert(60)
bst.insert(80)

print("In-order traversal:")
bst.inorder()
```

Output:-



Explanation :-

A Binary Search Tree (BST) is a tree data structure in which smaller values are stored in the left subtree and larger values are stored in the right subtree. In this implementation, the insert() method recursively adds new values to their correct positions. The in-order traversal method visits the left subtree, root, and right subtree, producing the elements in sorted order.

Task Description #5 – Priority Queue
Task: Use AI to implement a priority queue using Python’s heapq
module.
Sample Input Code:
class PriorityQueue:
pass
Expected Output:
 Implementation with enqueue (priority), dequeue (highest priority),
and display methods.

Input :-

```
import heapq


class PriorityQueue:
    """A priority queue implementation using heapq."""

    def __init__(self):
        """Initialize an empty priority queue."""
        self.queue = []

    def enqueue(self, item, priority):
        """Add an item to the priority queue with a priority."""
        heapq.heappush(self.queue, (priority, item))

    def dequeue(self):
        """Remove and return the highest-priority item."""
        if not self.queue:
            raise IndexError("Priority queue is empty.")

        priority, item = heapq.heappop(self.queue)
        return item

    def display(self):
```

```
        """Display all items in priority order."""
        for priority, item in sorted(self.queue):
            print("Priority:", priority, "Item:", item)


# Example usage
pq = PriorityQueue()

pq.enqueue("Task A", 3)
pq.enqueue("Task B", 1)
pq.enqueue("Task C", 2)

print("Priority Queue:")
pq.display()

print("Dequeued item:", pq.dequeue())

print("Priority Queue after dequeue:")
pq.display()
```

Output :-

```
PS C:\Users\HI\Desktop\rishwanth.ai assistant> cd 'c:\Users\HI\Desktop\rishwanth.ai
assistant'; & 'C:\Users\HI\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users
\HI\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launch
er' '56406' '--' 'c:\Users\HI\Desktop\rishwanth.ai assistant\Assignment11.py'
Priority Queue:
Priority: 1 Item: Task B
Priority: 2 Item: Task C
Priority: 3 Item: Task A
Dequeued item: Task B
Priority Queue after dequeue:
Priority: 2 Item: Task C
Priority: 3 Item: Task A
PS C:\Users\HI\Desktop\rishwanth.ai assistant>
```

Explanation :-

A Priority Queue is a data structure in which elements are processed according to their priority. This implementation uses Python's heapq module to maintain the priority queue. The enqueue() method adds an item with a priority, the dequeue() method removes the highest-priority item, and the display() method displays the items according to their priority.

Task Description #6 – Deque

Task: Use AI to implement a double-ended queue using
collections.deque.
Sample Input Code:
class DequeDS:pass
Expected Output: Insert and remove from both ends with docstrings.

Input :-

```
from collections import deque


class DequeDS:
    """A double-ended queue implementation using collections.deque."""

    def __init__(self):
        """Initialize an empty deque."""
        self.items = deque()

    def insert_front(self, item):
        """Insert an item at the front of the deque."""
        self.items.appendleft(item)

    def insert_rear(self, item):
        """Insert an item at the rear of the deque."""
        self.items.append(item)

    def remove_front(self):
        """Remove and return the item from the front."""
        if not self.items:
            raise IndexError("Deque is empty.")
        return self.items.popleft()
```

```
    def remove_rear(self):
        """Remove and return the item from the rear."""
        if not self.items:
            raise IndexError("Deque is empty.")
        return self.items.pop()

    def display(self):
        """Display all elements of the deque."""
        print(list(self.items))


# Example usage
dq = DequeDS()

dq.insert_front(20)
dq.insert_front(10)
dq.insert_rear(30)
dq.insert_rear(40)

print("Deque after insertion:")
dq.display()

print("Removed from front:", dq.remove_front())
print("Removed from rear:", dq.remove_rear())

print("Deque after removal:")
dq.display()
```

Output :-

```
PS C:\Users\HI\Desktop\rishwanth.ai assistant> cd 'c:\Users\HI\Desktop\rishwanth.ai--
assistant'; & 'C:\Users\HI\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users\
\HI\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher'
 '56465' '--' 'c:\Users\HI\Desktop\rishwanth.ai assignment1.py'
Deque after insertion:
[10, 20, 30, 40]
Removed from front: 10
Removed from rear: 40
Deque after removal:
[20, 30]
PS C:\Users\HI\Desktop\rishwanth.ai assistant>
```

Explanation :-

A Priority Queue is a data structure in which elements are processed according to their priority. This implementation uses Python's heapq module to maintain the priority queue. The enqueue() method adds an item with a priority, the dequeue() method removes the highest-priority item, and the display() method displays the items according to their priority.

Task Description #7 Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.

4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

->For each feature, select the most appropriate data structure from the list below:

1.Stack
2.Queue
3.Priority Queue
4.Linked List
5.Binary Search Tree (BST)
6.Graph
7.Hash Table
8.Deque

->Justify your choice in 2–3 sentences per feature.

->Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

->A table mapping feature → chosen data structure → justification.

->A functional Python program implementing the chosen feature with comments and docstrings.

Input :-

1. Data Structure Selection

Feature	Data Structure	Justification
Student Attendance Tracking	Hash Table	A Hash Table can associate each student ID with attendance information. It provides fast average-case lookup and updating of a student's attendance record.
Event Registration System	Binary Search Tree (BST)	A BST can store participant IDs in an organized manner and support searching and removal. This makes it suitable for managing registered participants.
Library Book Borrowing	Priority Queue	A Priority Queue can be used when books or borrowing requests need to be handled according to due-date priority. Earlier due dates can be given higher priority.
Bus Scheduling System	Graph	A Graph can represent bus stops as vertices and connections between stops as edges. This makes it suitable for representing bus routes and stop connections.
Cafeteria Order Queue	Queue	A Queue follows FIFO (First In, First Out) behavior. Therefore, students' cafeteria orders can be served in the same order in which they arrive.

Input :-

```
class CafeteriaQueue:
    """A queue for managing cafeteria orders using FIFO."""

    def __init__(self):
        """Initialize an empty cafeteria order queue."""
        self.orders = []

    def enqueue(self, order):
        """Add a new order to the rear of the queue."""
        self.orders.append(order)

    def dequeue(self):
        """Remove and return the oldest order from the queue."""
        if not self.orders:
            raise IndexError("No orders available.")
        return self.orders.pop(0)
```

```
    def peek(self):
        """Return the next order without removing it."""
        if not self.orders:
            raise IndexError("No orders available.")
        return self.orders[0]

    def size(self):
        """Return the number of waiting orders."""
        return len(self.orders)


# Example usage
cafeteria = CafeteriaQueue()

cafeteria.enqueue("Order 101 - Tea")
cafeteria.enqueue("Order 102 - Sandwich")
cafeteria.enqueue("Order 103 - Fried Rice")

print("Next order:", cafeteria.peek())
print("Served:", cafeteria.dequeue())
print("Next order:", cafeteria.peek())
print("Waiting orders:", cafeteria.size())
```

Output :-

```
PS C:\Users\HI\Desktop\rishwanth.ai assistant> cd 'c:\Users\HI\Desktop\rishwanth.ai assistant'; & 'C
:\Users\HI\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users\HI\.vscode\extensions\ms-
-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '56015' '--' 'c:\Users\HI\Desktop\
rishwanth.ai assistant\Aisignment11.py'
Nearest order: Order 181
Served: Indian Tea in 10s
Next order: Order 182  Sandwich
Waiting numbers: 7
PS C:\Users\HI\Desktop\rishwanth.ai assistant>
```

Explanation :-

Task 7 applies data structures to a Campus Resource Management System. Different data structures are selected according to the requirements of student attendance tracking, event registration, library book borrowing, bus scheduling, and cafeteria order processing. One selected feature, Cafeteria Order Queue, is implemented using a Queue because cafeteria orders can be processed in FIFO (First In, First Out) order. The program uses enqueue() to add orders and dequeue() to serve the oldest order first.

Task Description #8: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.

4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task:

-> For each feature, select the most appropriate data structure from the list below:

1.Stack
2.Queue
3.Priority Queue
4.Linked List
5.Binary Search Tree (BST)
6.Deque

->Justify your choice in 2–3 sentences per feature.
->Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

->A table mapping feature → chosen data structure → justification.
->A functional Python program implementing the chosen feature with comments and docstrings.

1. Data Structure Selection

Feature	Data Structure	Justification
Shopping Cart Management	Deque	A Deque allows products to be added and removed efficiently from both ends. This is useful for dynamically managing products in the shopping cart.
Order Processing System	Queue	A Queue follows FIFO (First In, First Out) behavior. Therefore, orders can be processed in the same order in which they were placed.
Top-Selling Products Tracker	Priority Queue	A Priority Queue can arrange products according to their sales priority. Products with higher sales counts can be retrieved according to the selected priority.

Feature	Data Structure	Justification
Product Search Engine	Binary Search Tree (BST)	A BST can organize product IDs in sorted order and support searching using comparisons. This provides an organized tree-based method for product lookup.
Delivery Route Planning	Linked List	A Linked List can represent a sequence of delivery locations. Each location can point to the next location in the delivery sequence.

2. Selected Feature for Implementation

We'll implement:

Shopping Cart Management → Deque

AI Prompt Used :-

Create a Python shopping cart using collections.deque. Implement add_product, remove_product, view_cart, and is_empty methods. Include comments and docstrings.

Input :-

```
from collections import deque


class ShoppingCart:
    """A shopping cart implemented using a double-ended queue."""

    def __init__(self):
        """Initialize an empty shopping cart."""
        self.products = deque()

    def add_product(self, product):
        """Add a product to the shopping cart."""
        self.products.append(product)

    def remove_product(self):
        """Remove and return a product from the front of the cart."""
        if not self.products:
            raise IndexError("Shopping cart is empty.")
        return self.products.popleft()

    def view_cart(self):
        """Display all products currently in the cart."""
```

```
        print("Cart:", list(self.products))

    def is_empty(self):
        """Check whether the shopping cart is empty."""
        return len(self.products) == 0


# Example usage
cart = ShoppingCart()

cart.add_product("Laptop")
cart.add_product("Mouse")
cart.add_product("Keyboard")

print("Shopping Cart:")
cart.view_cart()

print("Removed product:", cart.remove_product())

print("Shopping Cart after removal:")
cart.view_cart()

print("Is cart empty?", cart.is_empty())
```

Output :-

```
PS C:\Users\HI\Desktop\rishwanth.ai assistant> cd 'c:\Users\HI\Desktop\rishwanth.ai
assistant'; & 'C:\Users\HI\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users\
HI\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher'
'56667' '--' 'c:\Users\HI\Desktop\rishwanth.ai assistant\Assignment11.py'
Shopping Cart:
Cart: ['Laptop', 'Mouse', 'Keyboard']
Removed product: Laptop
Shopping Cart after removal:
Cart: ['Mouse', 'Keyboard']
Is cart empty? False
PS C:\Users\HI\Desktop\rishwanth.ai assistant>
```

Explanation :-

Task 8 applies data structures to a Smart Online Shopping System. Appropriate data structures are selected for shopping cart management, order processing, top-selling products tracking, product searching, and delivery route planning. One selected feature, Shopping Cart Management, is implemented using a Deque because it supports insertion and removal of products from both ends. The program uses methods to add products, remove products, display the cart, and check whether the cart is empty.